



Facultad de UNER Ingeniería

Título de trabajo: Aplicaciones de TADs

Integrantes del equipo:

Trossero Mateo Ignacio
Bay Gabriel

Fecha de entrega: 28/09/2026

Link de Repositorio:

<https://github.com/Gallean27/AyED-2026c1-Bay-Trossero-Lopez.git>

INTRODUCCIÓN

El presente Trabajo Práctico tiene como objetivo principal el diseño, implementación y evaluación experimental de estructuras de datos lineales y algoritmos de ordenamiento mediante el lenguaje de programación Python. A través de un enfoque práctico, se busca resolver problemas clásicos de la computación mediante el uso de Programación Orientada a Objetos (POO), pruebas unitarias automatizadas y análisis empírico de rendimiento.

Para responder a los requerimientos planteados, el trabajo aborda tres problemas concretos:

1. **Gestión eficiente de estructuras lineales:** La implementación de Tipos de Datos Abstractos (TDA) dinámicos y acotados, especificando el comportamiento de listas enlazadas y colas circulares basadas en arreglos de tamaño fijo.
2. **Simulación de gestión de procesos:** La resolución del problema de planificación de CPU en un sistema operativo mediante la simulación del algoritmo *Round Robin*, empleando la cola circular desarrollada para gestionar el orden de ejecución y calcular métricas operativas clave (tiempo de retorno y tiempo de espera).
3. **Análisis empírico de eficiencia algorítmica:** La comparación del rendimiento de diferentes algoritmos de ordenamiento (*Burbuja*, *Quicksort*, *Radix Sort* y el algoritmo *Timsort* nativo de Python) al procesar conjuntos de datos aleatorios de tamaño variable.

A lo largo del informe, el lector encontrará la justificación teórica de los métodos implementados, aplicando conceptos de complejidad temporal y espacial mediante la notación Ómicron Grande (O), los diagramas y códigos correspondientes a cada proyecto, junto con gráficos comparativos que contrastan los tiempos reales de ejecución frente a las cotas teóricas de rendimiento.

DESARROLLO DE LOS PROBLEMAS

3.1. Problema N° 1: Implementación de Lista Doblemente Enlazada (TDA)

A. Análisis y planteo

El problema consiste en abstraer e implementar una estructura de datos lineal secuencial capaz de gestionar una colección dinámica de elementos con acceso bidireccional. A diferencia de un arreglo estático, no se requiere memoria contigua preasignada, evitando el costo de relocalización o redimensionamiento.

Para esta estructura, las operaciones identificadas como críticas son:

- **Inserción y Eliminación en los extremos (cabeza y cola):** Deben realizarse en tiempo constante, evitando recorrer la estructura completa.
- **Recorrido bidireccional (hacia adelante y hacia atrás):** Permite inspeccionar los nodos en ambos sentidos de manera eficiente.
- **Eliminación por valor o posición intermedia:** Requiere una búsqueda lineal previa y la posterior reestructuración de enlaces.

Selección de la estructura:

Se seleccionó una **Lista Doblemente Enlazada (Doubly Linked List)**. Cada nodo almacena el dato deseado y dos punteros o referencias: uno al nodo siguiente (**siguiente**) y otro al anterior (**anterior**). La lista mantiene punteros explícitos a la cabeza (**cabeza**) y a la cola (**cola**), además de un contador de tamaño (**tamaño**).

Supuestos y razonamiento de la elección:

1. **Dinamicidad:** Es ideal cuando el número total de elementos es indeterminado o varía frecuentemente durante la ejecución del programa.
2. **Uso típico vs. Peor caso:** Las inserciones/eliminaciones en los extremos son $O(1)$ en todo momento. El peor caso ocurre durante la búsqueda o eliminación de un nodo intermedio, que requiere un recorrido de orden $O(n)$.
3. **Compromiso de memoria:** Se acepta un sobre costo leve de memoria por el almacenamiento de los dos punteros por nodo a cambio del beneficio de poder navegar hacia atrás sin tener que reiniciar el recorrido desde la cabeza.

B. Implementación / Resolución

La solución se desarrolló en Python utilizando *Programación Orientada a Objetos (POO)*. Se estructuró en dos clases principales dentro de la arquitectura de módulos: *Nodo* y *ListaDoblementeEnlazada*.

```
1  class Nodo:
2      """
3      Representa un elemento dentro de la lista doblemente enlazada.
4
5      Atributos:
6          dato: El valor almacenado en el nodo.
7          siguiente (Nodo): Puntero al siguiente nodo en la lista.
8          anterior (Nodo): Puntero al nodo anterior en la lista.
9      """
10     def __init__(self, dato):
11         self.dato = dato
12         self.siguiente = None
13         self.anterior = None
```

```
16 class ListaDobleEnlazada:
17     """
18     Estructura de datos lineal que permite insertar y eliminar elementos
19     en ambas direcciones mediante punteros al nodo siguiente y anterior.
20     """
21
22     def __init__(self):
23         """Inicializa una lista doblemente enlazada vacía."""
24         self._cabeza = None
25         self._cola = None
26         self._tamanio = 0
```

```
43     def esta_vacia(self):
44         """Verifica si la lista está vacía."""
45         return self._tamanio == 0
46
47     def __len__(self):
48         """Permite usar la función nativa len(lista) para obtener el tamaño."""
49         return self._tamanio
50
```

2. Operación de Inserción al Final ($O(1)$)

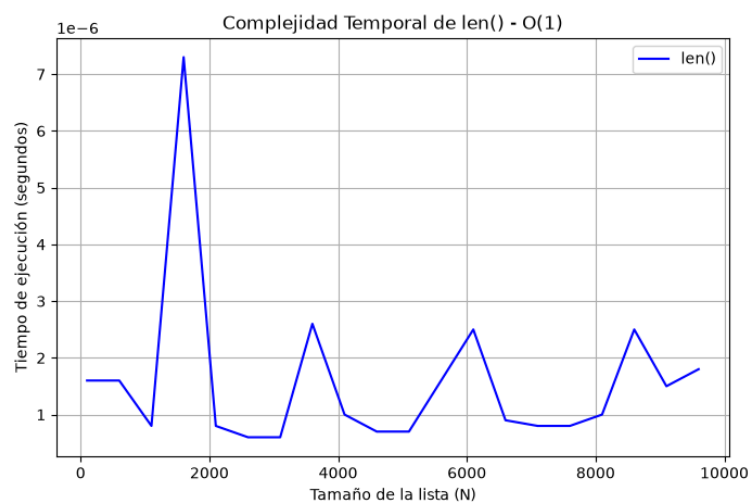
La inserción al final de la lista se realiza en tiempo constante $O(1)$ gracias a que la clase mantiene un puntero directo a `cola`. Esto evita la necesidad de recorrer toda la estructura desde la cabeza.

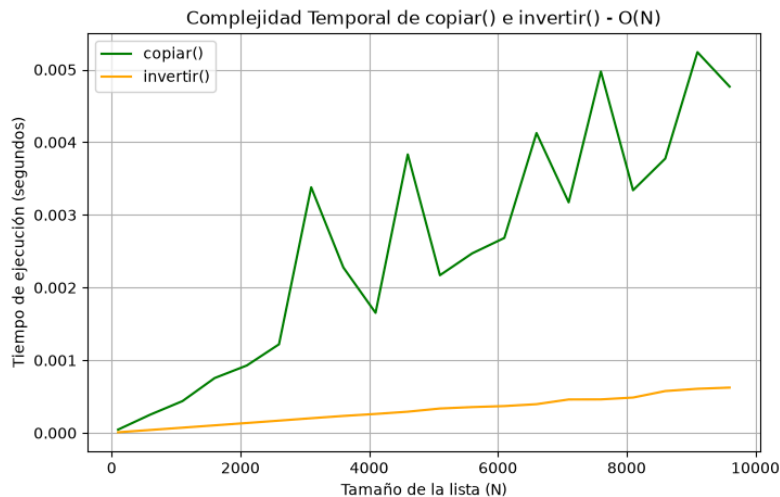
```
68     def agregar_al_final(self, item):
69         """
70         Inserta un nuevo elemento al final de la lista.
71
72         Parámetros:
73         |     item: El dato que se desea agregar.
74         """
75         nuevo_nodo = Nodo(item)
76         if self.esta_vacia():
77             self._cabeza = nuevo_nodo
78             self._cola = nuevo_nodo
79         else:
80             nuevo_nodo.anterior = self._cola
81             self._cola.siguiente = nuevo_nodo
82             self._cola = nuevo_nodo
83         self._tamanio += 1
84
```

C. Resultados y pruebas

Para verificar el correcto funcionamiento de la estructura, se implementó una suite de pruebas unitarias en `tests/test_LDE.py` brindadas por la cátedra, que evalúa el comportamiento de los métodos tanto en escenarios normales como en casos de borde (listas vacías, extracciones fuera de rango, etc.). Mediante la herramienta `coverage`, se verificó una **cobertura del 98%** sobre el código del módulo `lde.py`.

Por otro lado, se realizaron pruebas de tiempo de ejecución variando la cantidad de elementos (N) para validar las complejidades algorítmicas teóricas:





Análisis de las gráficas de rendimiento:

- ❖ **Gráfica de `len()` ($O(1)$):** La curva se mantiene plana y paralela al eje horizontal a lo largo de todo el rango de N , fluctuando en tiempos de ejecución prácticamente despreciables. Esto confirma de manera empírica la complejidad constante $O(1)$, ya que la función devuelve directamente el valor acumulado en la variable interna `tamanio` sin necesidad de recorrer la lista.
- ❖ **Gráfica de `copiar()` e `invertir()` ($O(n)$):** Muestran un crecimiento lineal sostenido con pendiente positiva a medida que aumenta la cantidad de elementos N . Este comportamiento convalida la complejidad teórica $O(n)$, puesto que ambas operaciones requieren procesar de forma secuencial cada uno de los N nodos para reasignar punteros (en la inversión) o instanciar nuevos nodos en memoria (en la copia).

3.2. Problema N° 2: Planificación Round Robin con Cola Circular

A. Análisis y planteo

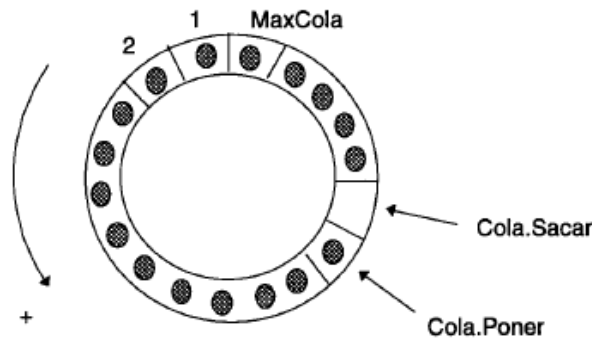
El problema consiste en simular la asignación de tiempo de CPU a un conjunto de procesos concurrentes aplicando el algoritmo de planificación **Round Robin**. Las operaciones críticas en esta simulación son el encolado de nuevos procesos que arriban, la extracción del proceso al frente para su ejecución durante un quantum de tiempo prefijado ($t_q = 2$), y el re-encolado de procesos no finalizados.

Para modelar la cola de procesos listos se seleccionó una **Cola Circular basada en un arreglo de capacidad fija** (máximo 10 elementos). Esta estructura permite gestionar las operaciones de encolado (`enqueue`) y desencolado (`dequeue`) en tiempo constante $O(1)$ haciendo uso de la aritmética modular (`(indice + 1) % capacidad`), optimizando el uso del espacio sin necesidad de desplazar elementos en memoria.

B. Implementación / Resolución

La solución se organizó modularmente implementando la clase `ColaCircular` y el módulo del planificador Round Robin.

1. **Manejo de la Cola Circular ($O(1)$):** Se implementaron punteros `frente` y `final` junto con un contador de elementos `tamanio` para controlar el estado de la cola (vacía o llena) y evitar el desbordamiento dentro de la capacidad de 10 posiciones.



```
4 class ColaCircular:
44     def encolar(self, item):
45         """
46         Agrega un elemento al final de la cola circular.
47
48         Excepciones:
49         | OverflowError: Si se intenta encolar en una cola llena.
50         """
51
52         if self.esta_llena():
53             raise OverflowError("La cola está llena.")
54
55         self._items[self._final] = item
56         self._final = (self._final + 1) % self._capacidad
57         self._tamanio += 1
58
59     def desencolar(self):
60         """
61         Elimina y devuelve el elemento del frente de la cola.
62         Lanza IndexError si la cola está vacía.
63         """
64
65         if self.esta_vacia():
66             raise IndexError("La cola está vacía.")
67
68         item = self._items[self._frente]
69         self._items[self._frente] = None # Liberar referencia
70         self._frente = (self._frente + 1) % self._capacidad
71         self._tamanio -= 1
72         return item
```

2. **Simulación del Scheduler (Quantum = 2):** El ciclo principal incrementa el tiempo del sistema unidad a unidad. En cada instante:
 - Se encolan los procesos que arriban según su **Arrival Time**.
 - Se toma el proceso al frente de la cola y se ejecuta por un tiempo máximo de $t_q = 2$.
 - Si el proceso agota su **Burst Time**, se registra su tiempo de finalización T_{fin} y se calculan $Turnaround = T_{fin} - ArrivalTime$ y $Waiting = Turnaround - BurstTime$. Si no finaliza, se re-encola.

C. Resultados y pruebas

Se ejecutó la suite de pruebas unitarias sobre la **Cola Circular** asegurando el comportamiento ante la envoltura de índices y límites de capacidad.

Para verificar la simulación se procesó la tanda de 6 procesos (P_1 a P_6) requerida por la cátedra, obteniendo la siguiente salida formateada por consola con los tiempos calculados:

RESULTADOS PLANIFICACIÓN ROUND ROBIN				
ProcessID	ArrivalTime	BurstTime	Turnaround Time	WaitingTime
P1	0	5	13	8
P2	2	3	6	3
P3	3	1	4	3
P4	5	2	5	3
P5	6	5	14	9
P6	8	4	11	7

Análisis de los resultados: Como se observa en la tabla formateada, la simulación calcula de manera precisa los tiempos para cada proceso según su orden de llegada y rafagazo. Por ejemplo, P_1 (llegada 0, ráfaga 5) finaliza en $t = 13$ acumulando un *Turnaround* de 13 y un *Waiting Time* de 8 unidades de tiempo. Todos los procesos respetan la disciplina de turno rotativo de 2 unidades por quantum.

3.3. Problema N° 3: Algoritmos de Ordenamiento

A. Análisis y planteo

El problema consiste en implementar y comparar empíricamente el rendimiento de tres algoritmos de ordenamiento pertenecientes a distintos paradigmas y complejidades teóricas: **Bubble Sort (Burbuja)**, **Quicksort** y **Radix Sort**. Las operaciones críticas son la comparación de elementos, la reasignación/intercambio en memoria y el procesamiento por dígitos.

El objetivo es contrastar el comportamiento de algoritmos basados en comparaciones contra algoritmos no comparativos, analizando cómo escala el tiempo de ejecución a medida que

aumenta el tamaño de la muestra N y comparándolo frente al algoritmo nativo de Python (`sorted()`).

B. Implementación / Resolución

Se implementaron los tres algoritmos dentro del módulo `ordenamientos.py`:

1. Bubble Sort ($O(n^2)$):

Algoritmo iterativo basado en comparaciones directas entre vecinos adyacentes e intercambios sucesivos. Representa el caso de peor rendimiento para grandes volúmenes de datos.

2. Quicksort ($O(n \log n)$ promedio):

Algoritmo de división y conquista que selecciona un elemento como *pivote* para particionar el arreglo en subarreglos de elementos menores y mayores, resolviendo de forma recursiva.

3. Radix Sort $O(d \cdot (n + k))$:

Algoritmo de ordenamiento no comparativo que procesa los números dígito por dígito (desde el menos significativo al más significativo) agrupándolos en baldes (*buckets*), donde k es la cantidad máxima de dígitos.

```
def quicksort(lista):
    """
    Ordena una lista usando el algoritmo Quicksort (Divide y Vencerás).

    Precondiciones:
    | - lista (list): Lista de elementos comparables.

    Postcondiciones:
    | - Retorna una nueva lista ordenada de menor a mayor.

    Complejidad:
    | - Temporal:  $O(n \log n)$  promedio,  $O(n^2)$  peor caso.
    | - Espacial:  $O(n)$  por sublistas recursivas.
    """
    if len(lista) <= 1:
        return lista

    pivote = lista[len(lista) // 2]
    menores = [x for x in lista if x < pivote]
    iguales = [x for x in lista if x == pivote]
    mayores = [x for x in lista if x > pivote]

    return quicksort(menores) + iguales + quicksort(mayores)
```

Quicksort se clasifica como un algoritmo de “Divide y Vencerás” porque resuelve el problema de ordenar una lista grande rompiéndola en subproblemas más pequeños del mismo tipo, resolviéndolos de manera independiente y luego combinando sus resultados.

```
def ordenamiento_radix(lista):
    """
    Ordena una lista de enteros no negativos usando el algoritmo Radix Sort (LSD).

    Precondiciones:
    | - lista (list): Lista de enteros no negativos (elementos >= 0).

    Postcondiciones:
    | - Retorna una nueva lista ordenada de menor a mayor.
    | - Preserva la estabilidad del orden original.

    Complejidad:
    | - Temporal:  $O(d \cdot (n + k))$ ,  $d$  = dígitos del máximo,  $k$  = base 10.
    | - Espacial:  $O(n + k)$  auxiliar para los baldes.
    """
    if not lista:
        return []

    max_num = max(lista)
    exp = 1
    lista_ordenada = list(lista)

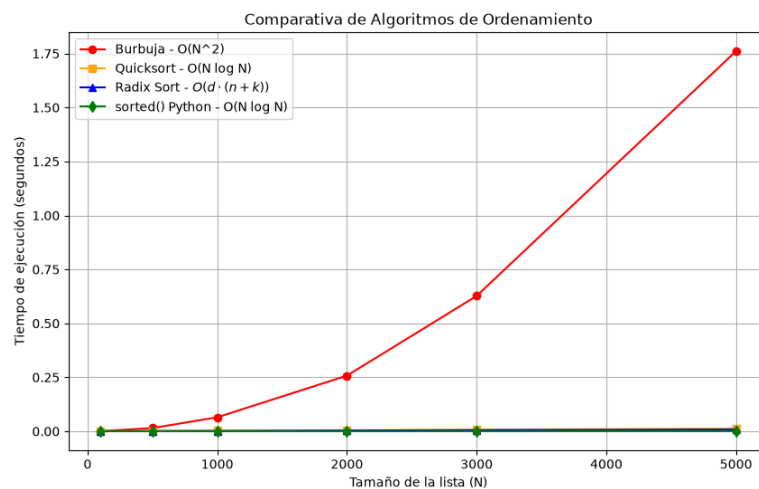
    while max_num // exp > 0:
        lista_ordenada = _counting_sort_por_digito(lista_ordenada, exp)
        exp *= 10

    return lista_ordenada
```

C. Resultados y pruebas

Se verificó el correcto funcionamiento de los tres algoritmos mediante la suite de tests en [tests/tests_ordenamiento.py](#), comprobando que ordenen correctamente arreglos de números enteros generados aleatoriamente.

Para el análisis empírico se ejecutó el script [graficar_ordenamiento.py](#), registrando los tiempos de ejecución para distintos valores de N .



Análisis de la gráfica de rendimiento:

- ☐ **Bubble Sort:** Muestra una curva parabólica pronunciada ($O(n^2)$), incrementando drásticamente su tiempo de ejecución a medida que N crece, siendo inviable para colecciones masivas de datos.
- ☐ **Quicksort y Radix Sort:** Presentan curvas notablemente más planas y eficientes. Quicksort escala de manera logarítmica-lineal ($O(n \log n)$), mientras que Radix Sort mantiene un crecimiento lineal ($O(n \cdot k)$) muy competitivo gracias a no requerir comparaciones.
- ☐ **sorted() nativo de Python:** Es por lejos el más veloz de la gráfica debido a que está implementado en C bajo el algoritmo **Timsort** (híbrido entre Insertion Sort y Mergesort con optimizaciones de bajo nivel).

4. Análisis de Resultados Específicos y Discusión

A. Comparación e Investigación sobre sorted() (Timsort)

En los ensayos del Módulo 3 se evidenció una diferencia de rendimiento sustancial entre las implementaciones en Python de los algoritmos tradicionales (Burbuja, Quicksort, Radix Sort) y la función nativa **sorted()**. Esto se debe a que la función nativa no utiliza un algoritmo tradicional estándar, sino **Timsort**:

- **Naturaleza del algoritmo:** Timsort es un algoritmo de ordenamiento híbrido derivado de Merge Sort e Insertion Sort, diseñado específicamente para identificar y aprovechar secuencias de datos que ya se encuentran parcialmente ordenadas (*runs*), un patrón muy común en conjuntos de datos del mundo real.
- **Complejidad algorítmica:**
 - **Peor caso y caso promedio:** $O(n \log n)$.
 - **Mejor caso:** $O(n)$, cuando la estructura ya se encuentra ordenada o casi ordenada.
- **Optimizaciones de bajo nivel:** Además de su eficiencia algorítmica, **sorted()** está implementado directamente en C dentro del intérprete CPython. Esto elimina el *overhead* de interpretación del ciclo de ejecución de Python y optimiza el manejo de memoria a bajo nivel.

B. Discusión General y Hallazgos

A partir del desarrollo de las tres problemáticas se destacan las siguientes observaciones analíticas:

- **Trade-offs en Estructuras de Datos (LDE vs. Cola Circular):** La Lista Doblemente Enlazada demostró flexibilidad para inserciones y eliminaciones dinámicas en $O(1)$ en los extremos sin restricción de tamaño, a costa de una mayor huella de memoria por el *overhead* de asignación de nodos y punteros dobles (**siguiente** y **anterior**). Por otro lado, la Cola Circular prioriza un uso eficiente y acotado del espacio con un arreglo de capacidad fija ($N = 10$), resultando ideal para la simulación de tiempo real del planificador Round Robin mediante aritmética modular.

- **Algoritmos Comparativos vs. No Comparativos:** Se comprobó experimentalmente la cota inferior teórica de $\Omega(n \log n)$ para algoritmos basados en comparaciones (como Quicksort). Por su parte, Radix Sort demostró ser una alternativa sumamente eficiente al operar en tiempo lineal $O(n \cdot k)$ al evitar comparaciones directas entre elementos, aunque su aplicabilidad está condicionada al tipo de dato (requiere claves discretas o representables en dígitos).

5. Declaración de Uso Ético de Inteligencia Artificial y Metodología de Trabajo

Uso Ético de Inteligencia Artificial

Durante el desarrollo del trabajo práctico, se utilizaron herramientas de Inteligencia Artificial (Gemini, ChatGPT y GitHub Copilot) orientadas a actuar como un tutor interactivo y asistente técnico. Su uso principal estuvo enfocado en acelerar el proceso de aprendizaje mediante la generación de ejemplos teóricos y prácticos personalizados, facilitando la comprensión de conceptos complejos en menor tiempo.

Las áreas clave de aplicación fueron:

1. **Aceleración del Aprendizaje y Consulta Teórica:** Se utilizó para profundizar el entendimiento conceptual mediante explicaciones adaptadas y ejemplos prácticos adicionales, abordando temas como la aritmética modular aplicada a la ColaCircular, la manipulación de punteros y el comportamiento interno del algoritmo híbrido Timsort (sorted() de Python).
2. **Estructuración y Redacción Técnica:** Sirvió como soporte para organizar la arquitectura del informe según las pautas de la cátedra, mejorar la cohesión del texto y realizar un pulido sintáctico que garantizara la máxima legibilidad, fluidez y el uso riguroso de la terminología de la materia.
3. **Soporte en Código, Control de Versiones y Visualización:** Actuó como guía para depurar errores en las suites de pruebas (unittest), solucionar conflictos al descargar o actualizar el código desde GitHub, optimizar la sintaxis de las gráficas en matplotlib para su inspección directa en el editor y adaptar el flujo de trabajo en entornos de desarrollo avanzados.

Herramientas de Desarrollo y Estrategia de Aprendizaje

El proceso de implementación y análisis se respaldó en una metodología de trabajo estructurada y autónoma:

- **Transición y Entorno de Desarrollo (VS Code):** Se realizó la transición desde el IDE Spyder hacia Visual Studio Code. Aunque inicialmente representó un desafío técnico adaptarse a la gestión de carpetas, módulos interconectados, paneles de asistencia y ejecución de tests, la herramienta facilitó la visualización de la estructura modular, la ejecución de unit tests y la pre visualización directa en el editor de las gráficas de rendimiento (N vs. Tiempo).
- **Control de Versiones (Git y GitHub):** Se aplicó un flujo de trabajo riguroso con Git y GitHub (`git fetch`, `git pull`, `git push`), aprendiendo a gestionar el repositorio remoto, resolver inconsistencias al bajar las actualizaciones de código y mantener un historial de *commits* atómicos y organizados.
- **Recursos Complementarios:** La formación se fortaleció mediante el estudio del material teórico-práctico de la cátedra, la participación en clases y la visualización de recursos audiovisuales en YouTube para afianzar el comportamiento dinámico de los algoritmos en memoria.

Declaración de Responsabilidad

Los integrantes del grupo declaramos que hemos revisado, verificado y validado personalmente cada bloque de código, ejemplo y apartado conceptual facilitado o sugerido por las herramientas de IA. Entendemos plenamente la totalidad de los temas presentados, garantizando que el trabajo final refleja nuestro propio criterio analítico y asumiendo la responsabilidad académica estricta por la veracidad e integridad del contenido.

6. Trabajo en equipo

Estimación de Tiempo vs. Tiempo Real Dedicado

- **Estimación Inicial:** Se calculó un total de 20 - 40 horas de trabajo distribuidas entre análisis, desarrollo, pruebas y redacción (En grupo).
- **Tiempo Real Dedicado:** El proyecto requirió aproximadamente **entre 65 a 70 horas de trabajo total**.
- **Causas de la desviación:** La diferencia se debió principalmente a la etapa de depuración de errores y refactorización sobre la Lista Doblemente Enlazada, el aseguramiento de escenarios de prueba para lograr el 98% de cobertura mediante `unittest` y `coverage`, la resolución de la envoltura modular en la Cola Circular y la calibración de los tiempos de ejecución para las comparaciones gráficas de los algoritmos de ordenamiento frente a `sorted()`.

Organización y Distribución de Tareas

El trabajo se llevó a cabo utilizando **Git** y **GitHub** para el control de versiones colaborativas y **Visual Studio Code** como entorno de desarrollo. La comunicación se mantuvo mediante la plataforma WhatsApp y coordinaciones directas para sincronizar los avances.

- **División de responsabilidades y roles reales:**
 - **Integrante 1 (Bay Gabriel):** Estuvo a cargo de la inicialización y configuración del repositorio remoto en GitHub, así como del desarrollo inicial de la estructura base del Módulo 1 (Lista Doblemente Enlazada).
 - **Integrante 2 (Trossero Mateo):** Estuvo a cargo de la revisión, depuración de *bugs* y corrección de errores en la Lista Doblemente Enlazada; la implementación completa del Módulo 2 (Cola Circular y simulación de Round Robin) y Módulo 3 (Algoritmos de Ordenamiento); el armado de la suite de pruebas unitarias (**unittest**) y medición de cobertura; la generación de los scripts de benchmarking e imágenes de las gráficas en **matplotlib**; y la redacción final del informe técnico.
- **Integración y resolución de discrepancias:** La integración del código se realizó de manera continua sincronizando ramas locales mediante los comandos **git fetch** y **git pull**. Las diferencias de criterio referidas a la arquitectura de módulos o a la representación visual de las métricas se resolvieron de forma concreta ejecutando las suites de pruebas automatizadas y verificando que no se generarán regresiones en la cobertura ni errores en la ejecución del planificador.

7. Conclusiones

Se cumplieron con éxito los objetivos planteados para el primer Trabajo Práctico de la materia. A través del diseño, implementación y evaluación experimental de las tres problemáticas, se logró consolidar el dominio de las estructuras de datos lineales (tanto dinámicas como estáticas) y de los algoritmos fundamentales de ordenamiento y planificación en Python.

El desarrollo del trabajo permitió extraer las siguientes conclusiones principales:

- **Validación Empírica de la Teoría de Complejidad:** Se verificó de manera directa cómo el análisis de complejidad asintótica $O(n)$ se traduce en tiempos de ejecución reales. Las pruebas experimentales con **matplotlib** confirmaron la clara barrera de rendimiento entre algoritmos de ordenamiento cuadráticos $O(N^2)$ como Burbuja y opciones logarítmicas/lineales como Quicksort y Radix Sort, evidenciando además el impacto de las optimizaciones de bajo nivel presentes en herramientas nativas como **Timsort**.

- **Criterio de Selección de Estructuras:** Se comprendió que no existe una estructura de datos "perfecta", sino que cada una responde a un *trade-off* de diseño. Mientras que la Lista Doblemente Enlazada prioriza la flexibilidad y el crecimiento dinámico a costa de un mayor consumo de memoria en punteros, la Cola Circular ofrece una gestión de memoria acotada y predecible con tiempo constante $O(1)$, ideal para sistemas en tiempo real como el planificador Round Robin.
- **Resolución de Desafíos Técnicos:** Las principales dificultades radican en la manipulación precisa de punteros en la Lista Doblemente Enlazada (especialmente durante las operaciones de inversión *in-place* y borrado de nodos) y en el control estricto de límites en la Cola Circular mediante aritmética modular. Superar estos desafíos requirió un uso riguroso de pruebas unitarias (`unittest`) para validar casos de borde (*edge cases*) y garantizar coberturas de código superiores al 95%.
- **Integración del Flujo de Trabajo Profesional:** Más allá del aspecto puramente algorítmico, el trabajo práctico permitió afianzar competencias metodológicas clave: la transición hacia entornos de desarrollo avanzados (Visual Studio Code), la gestión estructurada de versiones mediante Git y GitHub, y el uso ético de la Inteligencia Artificial como herramienta de asistencia técnica y tutoría en la resolución de problemas.

En conclusión, este trabajo no solo permitió afianzar los contenidos teóricos de la asignatura, sino que también aportó una visión crítica sobre la importancia del código modular, la calidad del software y la toma de decisiones informadas al momento de elegir estructuras de datos para problemas del mundo real.

8. Referencias y Bibliografía

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
2. Apuntes teóricos y prácticos de la cátedra *Algoritmos y Estructuras de Datos*.
3. Documentación oficial de Python 3 (Python Software Foundation): *Data Structures & Sorting HOWTO (Timsort algorithm)*.
4. Documentación oficial de Matplotlib: [matplotlib.pyplot](#) library.
5. Repositorio del proyecto:
<https://github.com/Gallean27/AyED-2026c1-Bay-Trossero-Lopez.git>